

Aritmética de Punto Fijo

Complemento A2, Signo y Tamaño

Representación con signo, crecimiento de bits, semántica de desplazamiento y trampas de contexto — con ejemplos numéricos

 Mayo 2026  IEEE 1800-2017  Fundación Fulgor / Tarpuy

1 · Complemento a 2 (CA2) — Representación con Signo

El complemento a 2 es el estándar universal para representar enteros con signo en hardware digital. Es la razón por la que en SystemVerilog la palabra clave `signed` cambia radicalmente cómo se interpretan y operan los bits.

1.1 — Interpretación de los bits

En CA2 de N bits, el MSB (bit más significativo) tiene **peso negativo**. El valor decimal se obtiene con:

$$\text{valor} = -b_{N-1} \cdot 2^{N-1} + b_{N-2} \cdot 2^{N-2} + \dots + b_1 \cdot 2 + b_0$$

El MSB es el **bit de signo**: si es 0 el número es positivo (o cero), si es 1 es negativo. El rango para N bits es $[-2^{N-1}, +2^{N-1}-1]$.

1.2 — Tabla de 4 bits

Bits (b ₃ b ₂ b ₁ b ₀)	Valor sin signo	Valor CA2 (con signo)	Cálculo CA2
0111	7	+7	-0·8 + 4+2+1
0110	6	+6	-0·8 + 4+2+0
0101	5	+5	
0100	4	+4	
0011	3	+3	
0010	2	+2	
0001	1	+1	
0000	0	0	-0·8 + 0
1111	15	-1	-1·8 + 4+2+1
1110	14	-2	-1·8 + 4+2+0
1101	13	-3	

Bits (b ₃ b ₂ b ₁ b ₀)	Valor sin signo	Valor CA2 (con signo)	Cálculo CA2
1100	12	-4	
1011	11	-5	
1010	10	-6	
1001	9	-7	
1000	8	-8	-1·8 + 0+0+0

1.3 — Propiedades clave

Rango asimétrico: hay un negativo más que positivos. En 4 bits: [-8, +7]. El +8 no existe en 4 bits con signo. Esta asimetría es la razón por la que $-8 \times -8 = 64$ requiere un bit extra (ver §6.2).

Para negar un número en CA2: invertir todos los bits y sumar 1.

EJEMPLO: -3 EN 4 BITS

+3 = 0011 ↓ invertir todos los bits 1100 ↓ sumar 1 -3 = 1101 → verificación: $-1 \cdot 8 + 4 + 0 + 1 = -3$
 ✓ Verificación cruzada con la tabla: 1101 → valor CA2 = -3 ✓

El bit de signo envuelve el círculo: sumar 1 al valor máximo +7 (0111) produce 1000 = -8. Esto es desbordamiento, no una operación válida.

2 · Notación de Punto Fijo: S(NB, NBF) y U(NB, NBF)

En punto fijo, un número binario es solo una secuencia de bits. **La representación** — cuántos bits hay, si hay signo, y dónde está el punto binario — determina completamente qué valor decimal corresponde a ese patrón de bits. El mismo patrón puede representar valores muy distintos según el formato elegido.

2.1 — La notación

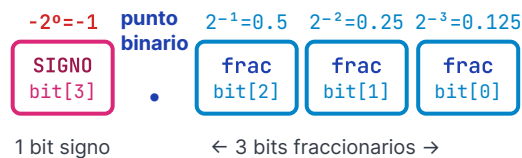
U(NB, NBF)**U** = Unsigned (sin signo)**NB** = Number of Bits (total)**NBF** = Number of Bits Fractional**NBI** = NB - NBF (bits enteros)Rango: $[0, 2^{NBI} - 2^{-NBF}]$ **S(NB, NBF)****S** = Signed (con signo, CA2)**NB** = Number of Bits (total)**NBF** = Number of Bits Fractional**NBI** = NB - NBF - 1 (bits enteros, sin contar el bit de signo)Rango: $[-2^{NBI}, 2^{NBI} - 2^{-NBF}]$

El -1 en NBI refleja que el bit de signo no contribuye al rango positivo — de ahí la asimetría CA2: un negativo más que positivos.

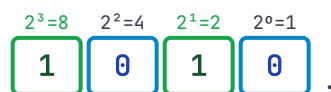
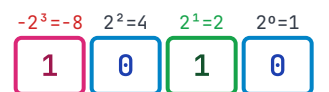
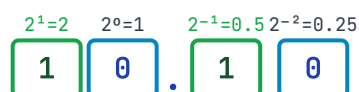
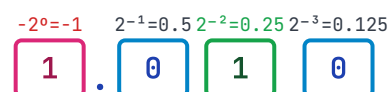
El punto binario es **implícito** — no existe en el hardware, solo en la interpretación. Todos los bits se almacenan como enteros; el diseñador decide mentalmente (y en el código) dónde está el punto.

2.2 — Layout de pesos por formato

El peso de cada bit depende de su posición relativa al punto binario. Para un formato de NB bits con NBF fraccionarios:

LAYOUT DE PESOS — EJEMPLO S(4,3): 4 BITS, 3 FRACCIONARIOS, CON SIGNO**2.3 — El mismo patrón de bits, cuatro interpretaciones**

El patrón 1010 de 4 bits vale algo distinto según el formato. El hardware almacena exactamente los mismos 4 bits en todos los casos.

U(4,0)Rango: $[0, 15]$ **S(4,0)**Rango: $[-8, +7]$ **U(4,2)**Rango: $[0, 3.75]$ **S(4,3)**Rango: $[-1, +0.875]$

```
// Mismo patrón de bits: 4'b1010 U(4,0): +10 rango [0, 15 ] resolución: 1 S(4,0): -6 rango [-8, +7 ] resolución: 1 U(4,2): +2.5 rango [0, +3.75 ] resolución: 0.25 S(4,3): -0.75 rango [-1, +0.875 ] resolución: 0.125
```

- i La **resolución** (el menor valor representable distinto de 0) es siempre $2^{-\text{NBF}}$. Agregar bits fraccionarios aumenta la resolución pero reduce el rango. Agregar bits enteros amplía el rango pero no cambia la resolución. El diseñador elige el tradeoff según el algoritmo.

Glosario de términos

LHS

Left-Hand Side — lado izquierdo de la asignación

```
resultado = a + b;
↑ LHS
```

RHS

Right-Hand Side — lado derecho de la asignación

```
resultado = a + b;
          ↑ RHS
```

MSB

Most Significant Bit — bit más significativo (izquierda)

```
1000 → MSB = 1
bit de signo en CA2
```

Toda expresión en SystemVerilog se evalúa bajo dos contextos determinados de forma independiente. Equivocarse en cualquiera de los dos produce corrupción silenciosa a nivel de bits — sin error de compilación ni advertencia por defecto.

2.1 — Contexto de Tamaño

El ancho en bits usado para el cómputo intermedio está determinado por el **operando más ancho** de la expresión, o por el LHS si es más ancho. Los operandos más angostos se extienden (con cero o con signo) antes de que la operación se ejecute.

2.2 — Contexto de Signo

Una expresión es **sin signo si alguno de sus operandos es sin signo**. Un único puerto logic sin la palabra clave signed convierte silenciosamente todo el cómputo a sin signo.

LRM §11.6.1: Si cualquier operando en una expresión con contexto determinado es sin signo, la expresión completa se evalúa como sin signo — incluyendo los operandos que fueron declarados individualmente como signed.

- i **LRM** = *Language Reference Manual* — el estándar oficial del lenguaje. Para SystemVerilog es el documento **IEEE 1800-2017**. Cada regla de comportamiento del lenguaje (cómo se evalúan expresiones, qué hace cada operador, cómo se resuelven los contextos de signo y tamaño) está definida ahí. Cuando se cita "LRM §X.Y.Z" se hace referencia a esa sección específica del estándar.

3 · Truncamiento por Asignación — Bits que se Pierden

En SystemVerilog, cuando se asigna una señal a otra de **menor ancho**, los bits sobrantes del lado de mayor peso se descartan silenciosamente. Esto es consecuencia directa de cómo se

declaran los rangos de bits: $[N:0]$ significa que el bit N es el MSB y el bit 0 es el LSB. Cuando el LHS es más angosto, solo se conservan los bits que entran — los de mayor índice se pierden.

```
logic [4:0] word_a;    // 5 bits: índices 4,3,2,1,0
logic [3:0] word_b;    // 4 bits: índices 3,2,1,0

assign word_a = 5'b0_1000;    // = 8 decimal
assign word_b = word_a;        // LHS tiene 4 bits → se guarda [3:0]
```

VISTA DE BITS — WORD_A → WORD_B

word_a =

0	1	0	0	0
[4]	[3]	[2]	[1]	[0]

 = 8

↓ assign word_b = word_a

word_b =

×	1	0	0	0
[4]	[3]	[2]	[1]	[0]

 = 8

 Bit descartado (fuera del LHS) Bit 1 conservado Bit 0 conservado

En este caso particular el resultado *numérico* es el mismo (8) porque el bit descartado era 0. Pero si $\text{word_a} = 5'b1_1000 = 24$, entonces word_b recibiría $4'b1000 = 8$ — se pierde el valor 16 que aportaba el bit [4], sin ninguna advertencia.

- La declaración $[N:0]$ en SV es **big-endian de índices**: el bit de mayor índice es el MSB. Al asignar a un LHS más angosto $[M:0]$ con $M < N$, SV conserva los bits de menor peso numérico y descarta los de mayor — los que no caben en el LHS.

3.1 — La endianness nombra los índices, no solo define el truncamiento

El punto más importante no es qué bit se descarta en una asignación — es que la declaración de endianness determina **qué índice corresponde a qué peso** en toda la señal. Cuando escribís $\text{signal}[2]$, estás accediendo a un bit de peso completamente diferente según cómo esté declarada la señal.

EL MISMO VALOR 8 — ÍNDICES CON DISTINTO SIGNIFICADO

logic [4:0] a = 5'b0_1000 — big-endian

$2^4=16$ $2^3=8$ $2^2=4$ $2^1=2$ $2^0=1$

0	1	0	0	0
a[4]	a[3] ★	a[2]	a[1]	a[0]

$a[3] = 1$ (peso 2^3)

logic [0:4] b = 5'b0_1000 — little-endian

$2^4=16$ $2^3=8$ $2^2=4$ $2^1=2$ $2^0=1$

0	1	0	0	0
b[0]	b[1] ★	b[2]	b[3]	b[4]

$b[3] = 0$ (peso 2^1)

A ambas señales se le asigna el mismo patrón de bits 01000, pero **a[3] = 1** mientras que **b[3] = 0**.

En big-endian [4:0], el índice 3 tiene peso $2^3 = 8 \rightarrow$ ahí vive el único 1.

En little-endian [0:4], el índice 3 tiene peso $2^1 = 2 \rightarrow$ ese bit es 0. El único 1 está en b[1] (peso $2^3 = 8$).

Esta diferencia se propaga a *todo* el código que usa esa señal: part-selects, condiciones if (signal[i]), conexiones a puertos, slices en cálculos. No es un problema de truncamiento — es un problema de semántica de nombre. Por eso mezclar ambas convenciones en el mismo diseño es una fuente segura de bugs silenciosos.

- ✗ La convención universal en RTL para ASIC es **[N:0] en todas las declaraciones**. El LRM permite ambas, pero las guías de estilo de cualquier equipo de diseño lo prohíben — no por capricho, sino porque un índice mal interpretado no genera error de compilación.

3.2 — Truncamiento: LHS más angosto que RHS

Cuando el LHS tiene menos bits que el RHS, SV descarta los bits que no caben — siempre los de mayor índice declarado, que corresponden a los de mayor peso en big-endian [N:0].

```
logic [4:0] wide  = 5'b1_1010; // = 26
logic [3:0] narrow;
assign narrow = wide;           // LHS 4b < RHS 5b → se pierde bit[4]
```

TRUNCAMIENTO BIG-ENDIAN [4:0] → [3:0]



El bit [4] que valía $2^4 = 16$ se descartó silenciosamente. El resultado almacenado es 1010 = 10, no 26. Esta es exactamente la misma razón por la que el LHS del multiplicador debe tener $W_A + W_B$ bits — si es más angosto, los bits de mayor peso del producto se pierden de la misma manera.

3.3 — Truncamiento little-endian [0:N]: se pierde el extremo opuesto

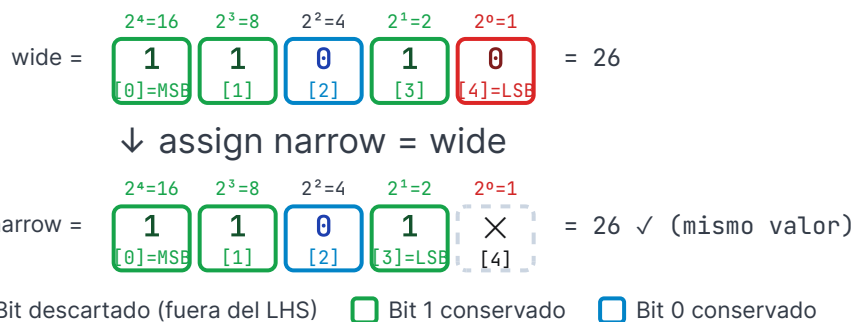
En little-endian [0:N] el índice 0 es el MSB (mayor peso). Al asignar a un LHS más angosto [0:M], SV conserva los índices [0:M] y descarta [M+1:N] — que ahora son los de **menor peso** (LSBs). Es lo contrario al caso big-endian.

```

logic [0:4] wide = 5'b1_1010; // = 26, [0]=MSB
logic [0:3] narrow;
assign narrow = wide;           // LHS 4b < RHS 5b → se pierde b[4] (LSB)

```

TRUNCAMIENTO LITTLE-ENDIAN [0:4] → [0:3]



En este ejemplo el bit perdido era 0 así que el valor no cambió. Pero si `wide = 5'b1_1011 (= 27)`, el bit [4] sería 1 (peso $2^0 = 1$) y narrow recibiría 1101 = 26 — se perdió la resolución de 1 LSB. En punto fijo esto equivale a un error de truncamiento de 2^{-NBF} .

5.1 — Costo de un sumador en hardware

El bloque atómico de cualquier sumador es el **Full Adder (FA)**: toma 3 bits de entrada (A, B, carry-in) y produce 2 bits de salida (suma, carry-out). Se implementa con aproximadamente 9 compuertas lógicas básicas.

Un sumador de N bits necesita N full adders encadenados — uno por posición de bit. El carry se propaga de izquierda a derecha, así que el retardo crece linealmente con N.

SUMADOR DE N BITS

Área: N full adders $\approx N \times 9$ compuertas Retardo: $O(N)$ (el carry del bit 0 debe llegar al bit N-1) Ejemplo – sumador de 16 bits: 16 FA ≈ 144 compuertas Ejemplo – sumador de 32 bits: 32 FA ≈ 288 compuertas (el doble)

5.2 — Costo de un multiplicador N × M bits

Multiplicar A (N bits) × B (M bits) equivale a sumar M copias de A, cada una desplazada según el peso del bit correspondiente de B. Eso genera **M filas de N bits** que hay que sumar todas.

CONSTRUCCIÓN DEL ARRAY MULTIPLIER

Paso 1 – productos parciales: M bits de B × N bits de A = $N \times M$ compuertas AND (una compuerta AND por cada par de bits $b_i \cdot a_j$) Paso 2 – sumar las M filas: Se necesitan M-1 sumas, cada una sobre N bits Cada suma de N bits cuesta N full adders Total árbol de suma: $(M-1) \times N \approx N \times M$ full adders Costo total: $N \times M$ compuertas AND + $N \times M$ full adders $\approx 2 \times N \times M$ compuertas Retardo:

$O(M)$ sumas encadenadas, cada una $O(N) \rightarrow O(N \times M)$

MULTIPLICADOR 8×8

AND gates: $8 \times 8 = 64$ Full adders: $7 \times 8 = 56$
Total: ~ 120 celdas Retardo: ~ 7 sumas de 8b

MULTIPLICADOR 16×16

AND gates: $16 \times 16 = 256$ Full adders: $15 \times 16 = 240$ Total: ~ 496 celdas Retardo: ~ 15 sumas de 16b Área $\sim 4\times$ respecto al 8×8

- ✗ El área crece **cuadráticamente**: duplicar el ancho de los operandos cuadruplica el área. Un multiplicador de 32×32 tiene $\sim 4\times$ el área de uno de 16×16 . Por eso reemplazar multiplicaciones por potencias de 2 con shifts (área = 0) es una optimización tan significativa cuando el algoritmo lo permite.

4.3 — Shift izquierdo = multiplicación por potencia de 2, sin costo de área

En notación binaria posicional, cada bit en la posición i vale 2^i . Desplazar todos los bits N posiciones a la izquierda mueve cada bit de la posición i a la posición $i+N$, multiplicando su valor por 2^N . Los huecos en el LSB se rellenan con **0**.

$$a \ll N \equiv a \times 2^N \text{ (inserta } N \text{ ceros por la derecha)}$$

EJEMPLO — $A = 3 = 4'B0011$, DESPLAZAR 2 POSICIONES A LA IZQUIERDA

Original: 0 0 **1 1** = 3 Desplazado: 0 0 **1 1** 0 0 = 12 = $3 \times 4 = 3 \times 2^2$ ✓ Hardware para implementar $a \ll 2$: **0 compuertas**. Es solo reasignar las conexiones del bus — puro ruteo.

4.4 — Tradeoff: área vs restricción

MULTIPLICADOR (BOOTH + WALLACE)

✓ Multiplica por cualquier valor ✓
Coeficiente variable en runtime ✓ Área reducida vs naïve ($\sim N^2/2$) X Sigue siendo costoso en área X Retardo y potencia significativos

SHIFT IZQUIERDO ($\ll N$)

✓ Solo potencias de 2: 1, 2, 4, 8... ✓
Coeficiente fijo en tiempo de diseño ✓ Área = 0 (solo ruteo) ✓ Retardo = 0 (propagación de señal) ✓ Sin consumo de potencia dinámico

El shift solo reemplaza al multiplicador cuando el factor de escala es una potencia de 2 **fija en tiempo de diseño**. Casos típicos en DSP: normalización posterior a una FFT (dividir por $N = 2^k$), alineación del punto binario después de una multiplicación, o escalado de ganancia con pasos binarios. En cambio, un filtro FIR con coeficientes arbitrarios necesita multiplicadores reales — ahí no hay atajos.

4.4 — Shift derecho: aquí sí importa el signo

Dividir por 2^N requiere desplazar a la derecha. Los bits salen por el LSB (se pierden, equivale al redondeo hacia $-\infty$) y entran bits nuevos por el MSB. A diferencia del desplazamiento

izquierdo, ahora la respuesta depende del signo: **¿qué bit entra por el MSB?**

- **>> — lógico:** rellena siempre con **0**. Correcto solo para sin signo. Para negativos en CA2 convierte el número en positivo.
- **>>> — aritmético:** rellena con el **bit de signo original**. Correcto para CA2 — preserva el signo y divide por 2^N .

$$a \ggg N \equiv a / 2^N \text{ (entero, redondeo hacia } -\infty, \text{ solo si } a \text{ es signed)}$$

EJEMPLO — A = -8 (4 BITS CA2) = 4'B1000, DIVIDIR POR 2

Original: 1 0 0 0 = -8 (MSB = 1 → negativo) **a >> 1 (lógico, entra 0 por el MSB):** entra → 0 | 1 0 0 0 → sale 0 resultado: 0 1 0 0 = +4 X (debería ser -4, el signo se destruyó) **a >>> 1 (aritmético, entra el bit de signo original):** entra → 1 | 1 0 0 0 → sale 0 resultado: 1 1 0 0 = -4 ✓

MAL — DESPLAZAMIENTO LÓGICO SOBRE DATO CON SIGNO

```
logic signed [15:0] coeff = -256;    // 0xFF00
logic      [15:0] scaled;
scaled = coeff >> 4;    // 0xFF0 = +4080 X (debería ser -16)
```

BIEN — DESPLAZAMIENTO ARITMÉTICO

```
logic signed [15:0] scaled;
scaled = coeff >>> 4;    // 0xFFF0 = -16 ✓
```

5 · Concatenación y Signo — Análisis Matemático

La concatenación {A, B} produce un resultado **sin signo por definición en el LRM**, independientemente de los tipos declarados de A y B. Los bits son correctos, pero el *tipo* es sin signo, lo que contamina cualquier expresión que use el resultado.

EJEMPLO — HI = -1, LO = 0 (16 BITS CADA UNO)

hi = 16'shFFFF = -1 bits: 1111_1111_1111_1111 lo = 16'sh0000 = 0 bits: 0000_0000_0000_0000 Como **signed 32 bits**: -65 536 ← lo que queremos Como **unsigned 32 bits**: 4 294 901 760 ← lo que SV ve (tipo sin signo) Operación: dividir por 2 (>>> 1) Esperado: -65 536 / 2 = -32 768 = 0xFFFF_8000 **Sin cast**: 4 294 901 760 / 2 = 2 147 450 880 = 0x7FFF_8000 X (MSB ← 0) **Con cast**: signed'({hi,lo}) >>> 1 = -32 768 = 0xFFFF_8000 ✓ (MSB ← 1)

- ¡ Una *asignación simple* full = {hi, lo} hacia un LHS declarado signed almacena los bits correctamente — la transferencia de bits es agnóstica al tipo. El error aparece cuando {hi,lo} se usa directamente en una expresión (desplazamiento, comparación, aritmética) donde el contexto de signo determina el comportamiento.

MAL — CONCATENACIÓN USADA DIRECTAMENTE EN EXPRESIÓN

```
logic signed [15:0] hi, lo;  
logic signed [31:0] half;  
half = {hi, lo} >>> 1; // {hi,lo} sin signo → rellena con 0 X
```

BIEN — CAST ANTES DE USAR EN EXPRESIÓN

```
half = signed'({hi, lo}) >>> 1; // desplazamiento aritmético ✓
```

6 · Crecimiento de Bits: Sumadores y Multiplicadores

El ancho de bits requerido en la salida se deriva directamente del rango posible de la señal. Calcularlo correctamente es la base de la aritmética de punto fijo sin pérdida.

6.1 — Ancho del sumador

Al sumar N valores con signo de W bits cada uno, la suma máxima es $N \times (2^{W-1}-1)$. Para no desbordar:

$$W_{\text{salida}} = W_{\text{entrada}} + \lceil \log_2(N) \rceil$$

EJEMPLO — SUMANDOS DE 4 BITS CON SIGNO, RANGO [-8, +7]

$N=2$: $\text{máx} = 7+7 = 14 \rightarrow$ necesita 5 bits ($4 + \lceil \log_2(2) \rceil = 4+1 = 5$) $N=4$: $\text{máx} = 7 \times 4 = 28 \rightarrow$ necesita 6 bits ($4 + \lceil \log_2(4) \rceil = 4+2 = 6$) $N=8$: $\text{máx} = 7 \times 8 = 56 \rightarrow$ necesita 7 bits ($4 + \lceil \log_2(8) \rceil = 4+3 = 7$)

6.2 — Ancho del multiplicador

Analizamos todos los casos extremos con **números de 4 bits** para ver exactamente cuántos bits necesita el producto.

Caso con signo (4 bits CA2): rango [-8, +7]

TODOS LOS PRODUCTOS EXTREMOS POSIBLES

$(+7) \times (+7) = +49 \rightarrow$ necesita 7 bits signed ($\text{máx } 7b = +63 \checkmark$) $(+7) \times (-8) = -56 \rightarrow$ necesita 7 bits signed ($\text{mín } 7b = -64 \checkmark$) $(-8) \times (+7) = -56 \rightarrow$ necesita 7 bits signed $(-8) \times (-8) = +64 \rightarrow$ máximo signed $7b = +63 \rightarrow$ **NO CABE en 7 bits** 2^7 signed $\rightarrow$ rango $[-64, +63]$: $64 > 63 \times 2^8$ signed $\rightarrow$ rango $[-128, +127]$: $64 \leq 127 \checkmark \rightarrow$ necesita **8 bits** Conclusión: el único caso que fuerza el bit extra es $(-8) \times (-8) = 64$. Si ese caso no puede ocurrir, bastan $4 + 3 = 7$ bits.

-8 × -8 ES POSIBLE

Rango incluye +64 +64 no cabe en 7 bits signed $W_A + W_B = 4+4 = 8$ bits (rango signed: $[-128, +127]$)

-8 × -8 NO ES POSIBLE

Peor caso: $\max(49, 56) = 56$ 56 cabe en 7 bits signed $[-64, +63]$ $W_A + (W_B - 1) = 4+3 = 7$ bits (1 bit menos, sin pérdida)

$$W_{\text{producto}} = W_A + W_B \text{ (caso general con signo)}$$

$$W_{\text{producto}} = W_A + W_B - 1 \text{ (si se garantiza que los mínimos no coinciden)}$$

6.3 — Qué pasa cuando el LHS es demasiado angosto: vista de bits

Si el LHS (la variable que recibe el resultado) no tiene suficientes bits, **los bits de mayor peso**

se descartan silenciosamente. El caso más ilustrativo es $-8 \times -8 = 64$ en $S(4,0)$: el resultado correcto necesita 8 bits, pero si se usa un LHS de 7 bits ($S(7,0)$), el bit 6 pasa a ser el bit de signo y 64 se interpreta como -64 .

$-8 \times -8 = 64$ — $S(4,0) \times S(4,0)$ NECESITA $S(8,0)$

64 completo $S(8,0)$:

	-2^7	$2^6=64$	2^5	2^4	2^3	2^2	2^1	2^0	
	0	1	0	0	0	0	0	0	= +64 ✓
	[7]	[6]	[5]	[4]	[3]	[2]	[1]	[0]	

↓ LHS de 7 bits — $S(7,0)$

LHS $S(7,0)$:

	-2^7	$-2^6=-64$	2^5	2^4	2^3	2^2	2^1	2^0	
	X	1	0	0	0	0	0	0	= -64 ✗
	[7]	[6]=SG	[5]	[4]	[3]	[2]	[1]	[0]	

Bit descartado Nuevo bit de signo (era entero) Bit 1 correcto

- ✗ No se descartó ningún bit — los 8 bits están todos ahí. El problema es que en $S(7,0)$ el bit [6] **es el bit de signo** (peso $-2^6 = -64$), no un bit entero positivo. El mismo patrón 0100_0000 que en $S(8,0)$ vale +64, en $S(7,0)$ vale -64. Este es exactamente el caso que fuerza $W_A + W_B = 8$ bits para $S(4,0) \times S(4,0)$.

★ Resumen

CA2: el MSB tiene peso negativo. Rango asimétrico $[-2^{N-1}, +2^{N-1}-1]$. El positivo máximo de 4 bits es +7, no +8.

Desplazamiento: >> rellena con 0 (lógico, sin signo). >>> replica el bit de signo (aritmético, divide por 2^N en CA2).

Concatenación: {a,b} es siempre sin signo. Usar signed' () antes de usarlo en expresiones.

Ancho del multiplicador: $W_A + W_B$ bits. El único caso que fuerza ese bit extra en CA2 es $(-2^{N-1})^2$ — para 4 bits, $-8 \times -8 = 64$ no cabe en 7 bits signed (+63 máx). Con LHS de ancho correcto, producto = $a * b$ es suficiente; no usar cast en el RHS.